EXECUTE THE PROGRAM: AN EXECUTION-BASED AUDIT OF MESH-TO-CAD RECONSTRUCTION AND AGENT TOOL USE

Anonymous authors
Paper under double-blind review

ABSTRACT

The output of mesh-to-CAD reconstruction is a program, so we judge systems by executing the program they deliver: we replay it in a sandbox, require every solid to be valid, and score the executed solid against the target, with failures scored zero. We apply this protocol in three prospectively specified studies on hard-tier CADBench parts. On 300 held-out parts, a released reconstruction optimizer obtains no valid replay on 41 parts; delivering the best program that actually executes raises validity-gated IoU by +0.104 (95% interval [+0.071, +0.138], the preregistered primary contrast), and three later stages add +0.028, +0.006 and +0.009. Under the same judge, Claude Opus 5.5 acting through a CAD harness with plain code scores 0.975 on 40 reserved held-out parts, above the improved optimizer's 0.951 (exploratory), while the optimizer's stages offered as agent tools did not raise its score and caused deadline failures. Profiling showed that the tool cost was implementation overhead. A revised tool package, developed on development parts only and evaluated on 40 new parts with three episodes per condition, halves Opus's time with the optimizer's tools (8.9 to 4.2 minutes per part) and leaves every Opus delivery valid; with it the optimizer's tools beat plain code by a small Holm-significant margin (+0.003), but the preregistered improvement of the package over the old tools (+0.037) is not established by its declared test.

1 INTRODUCTION

A mesh-to-CAD system delivers a program. A designer opens it, edits its sketches and features, and a CAD kernel turns it into a solid. Whether that solid exists, is valid, and matches the target is therefore the quantity of interest, and it is not the quantity most search procedures optimize. Optimization-based reconstructors such as CADFit (Nehme et al., 2026) fit extrusion and revolve candidates to slices of the target mesh, compose them with mesh booleans, and keep the composition with the highest volumetric IoU; the program is written out at the end. Language-model agents for CAD (Mallis et al., 2025; Savrasov et al., 2026) likewise act through tools whose outputs are geometry, not necessarily a program that builds.

We evaluate both kinds of system by the solid their delivered program builds (Section 3). The protocol is simple: execute the program in a sandbox, check OCC validity of the shape and of every solid, score the executed solid with a judge that does not depend on how clean its tessellation is, and score any failure as zero while keeping every part in the denominator. We fix each study's arms, cohort and analysis before running it, keep cohorts disjoint by model index, and record every amendment and rerun with its date and cause.

We ask three questions, each answered by its own study.

- **Q1** How much of a reconstruction optimizer's quality is lost between its search and its delivered program, and which repairs recover it on held-out parts? (Section 4)
- **Q2** How do frontier language-model agents compare with the optimizer under the same judge, and do the optimizer's stages help them as tools? (Section 5)

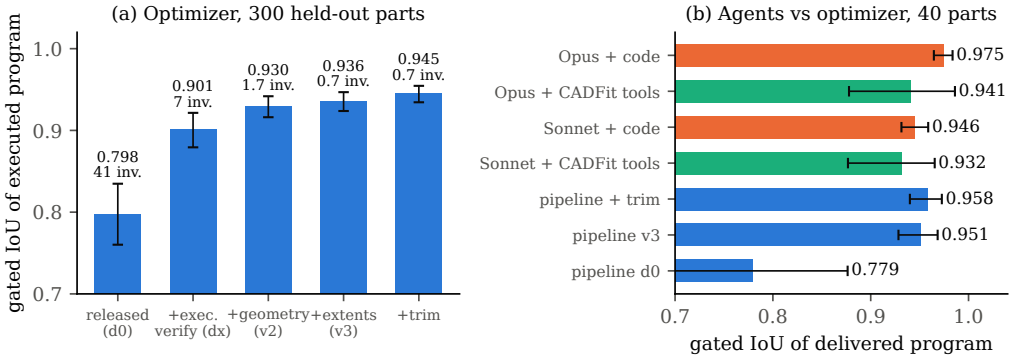


Figure 1: **(a)** The optimizer on 300 held-out fresh2 parts: validity-gated IoU of the executed program for the released pipeline and each added stage; three-seed arms are averaged within part; error bars are 95% split-stratified bootstrap intervals over parts; "inv." counts parts without a valid replay (mean over seeds). **(b)** The 40 reserved fresh2 parts: Claude agents (three episodes per part, each delivered program re-judged) and the optimizer arms on the same parts. Axes start at 0.7.

Q3 If those tools do not help, is the cause the geometry they compute or the cost of computing it? (Section 6)

Our answers (Figure 1):

1. **Execution-verified delivery is the largest single improvement.** The released optimizer obtains no valid replay on 41 of 300 held-out parts (23 built but invalid, 7 build errors, 1 missing, 10 over the replay resource limits). Delivering the best candidate program that builds a valid solid raises validity-gated IoU by +0.104 [+0.071, +0.138] (preregistered primary). Geometry fixes, precise extents and a silhouette trim each add a smaller, significant increment.
2. **A frontier model with plain code matches the improved optimizer.** Claude Opus 5.5, writing CadQuery through a harness that executes every change, scores 0.975 on 40 reserved parts with 120/120 valid deliveries, against 0.951 for the optimizer's v3 (difference +0.024 [+0.003, +0.048], exploratory); superiority over the optimizer with trim is not established. Offering the optimizer's stages as tools did not raise the score and produced episodes that ran out of time with nothing delivered.
3. **The tool cost was overhead, and removing it changes the comparison.** Profiling attributed the cost to redundant computation in profile fitting and in repeated checking, not to the geometry. A revised package of exact rewrites plus a per-check time cap halves Opus's time with the optimizer's tools and leaves every Opus delivery valid; with it those tools beat plain code by +0.003 (Holm $p = 0.028$). Its preregistered quality improvement over the old tools, +0.037 [+0.002, +0.096], is not established by the declared sign test ($p = 0.121$).

2 Related Work

Mesh- and point-cloud-to-CAD. Learned methods map geometry to construction sequences: DeepCAD (Wu et al., 2021) and Point2Cyl (Uy et al., 2022) predict sketch-and-extrude structure, zone graphs (Xu et al., 2021) search over extrusion sequences, and CAD-Recode (Rukhovich et al., 2025) decodes point clouds into CadQuery code that is executed for evaluation. CADFit (Nehme et al., 2026) optimizes instead: it fits extrusion and revolve candidates to slices of the mesh and composes them greedily to maximize volumetric IoU. Datasets include ABC (Koch et al., 2019), Fusion 360 Gallery (Willis et al., 2021) and DeepCAD; CADBench (Doris et al., 2026) unifies them with face-count tiers and metrics for fidelity, executability and compactness. We do not propose a new reconstructor; we measure an optimizer by its executed program and test which repairs survive that measurement on held-out parts.

Table 1: Positioning. "Executed in eval": the reported score is computed on the solid built by running the returned program. "Held-out, frozen": arms and analysis fixed before a cohort drawn for the study.

Work	Input	Tools / proposals	Executed in eval	Held-out, frozen
CADFit	mesh	built-in pipeline stages	search scored on meshes	–
CAD-Recode	point cloud	none (learned decoder)	yes	–
CADReasoner	renders + points	trained iterative editor	yes	–
CADFather	mesh	learned + algorithmic tools	yes	–
This work	mesh	optimizer stages as tools	yes (sandboxed replay)	yes

Iterative and agentic CAD generation. Tool-using language models (Yao et al., 2023; Schick et al., 2023) and agent–computer interfaces (Yang et al., 2024) motivate CAD agents that act, observe and revise. CADReasoner (Kabisov et al., 2026) trains a model to edit a runnable CadQuery program from the geometric discrepancy of its previous output, and IterCAD (Hu et al., 2026) trains a multimodal agent against an executable CAD sandbox. CAD-Assistant (Mallis et al., 2025) plans FreeCAD actions with CAD-specific tools, and CADFather (Savrasov et al., 2026) coordinates learned and algorithmic proposal tools with a vision-language assistant, executes proposed programs and keeps the best valid one. We use frozen closed models without training, take the tools from one optimizer, and compare tool packages that differ only in implementation and checking policy on a cohort drawn after the packages were frozen.

3 Execution-based evaluation

Executed program. Every system delivers a CadQuery program (CadQuery contributors, 2024). We run it in a fresh sandboxed process with no network access and a 900 s limit (the trim arm of Section 4 is the one exception, noted there), take its result solid, and check OCC validity of the shape and of every solid it contains. A program that fails to build, builds an invalid solid, exceeds the limit, or is missing is a failed delivery.

Judge. The executed solid is tessellated and compared with the target, normalized so that its largest extent spans $[-1, 1]$. When both meshes are closed and consistently oriented, IoU is computed exactly with manifold booleans (Lambert & Manifold contributors, 2024); otherwise it is a seeded Monte-Carlo estimate in which inside-ness of the reconstruction is the generalized winding number (Jacobson et al., 2013), so a tessellation cracked by float round-off is not judged as a hollow shell. **Validity-gated IoU** is the IoU of the executed solid if the delivery succeeded and 0 otherwise. Every cohort part stays in every denominator.

Cohorts. All parts are hard-tier CADBench (Doris et al., 2026) solids from two splits, benchA and benchE, drawn by seeded shuffles that exclude every part whose model index (the first field of its file id) appears in any earlier set. Development used separate sets (Appendix A). **fresh2** (150 + 150 parts) was drawn after development for the optimizer study; **40 reserved parts** (20 + 20) were drawn from fresh2's manifest before any fresh2 outcome existed and used for the agent study; **fresh3** (20 + 20) was drawn after the revised tools and their protocol were frozen. The optimizer's outcomes on fresh2's other 260 parts were seen before the agent studies began; fresh3 is the only cohort unseen by every analysis.

Statistics. All comparisons are paired over parts. Repeated runs or episodes are averaged within part first. Intervals are nominal 95% bootstrap intervals that resample parts within split (Efron & Tibshirani, 1993). Tests are exact two-sided sign tests on per-part differences; a part counts as better or worse when its difference exceeds ± 0.005 in the optimizer study and $\pm 10^{-9}$ in the agent studies, and as neutral otherwise. Holm correction is applied within each declared family (stated per study).

Preregistration and amendments. Each study's arms, cohort, primary and secondary comparisons and analysis code were committed before its runs. Changes made after outcomes existed are recorded as dated amendments, reported in the main text with their effect, and listed with every exclusion and rerun in Appendix D.

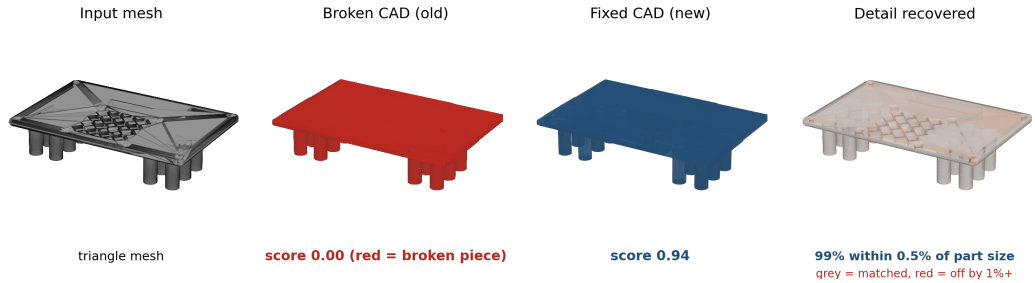


Figure 2: A development part (not from any evaluation cohort). Left to right: the input mesh; the program written by the optimizer's search, which stacks 208 overlapping solids of which 34 are OCC-invalid (red) and therefore scores 0; the program delivered by the execution check, a fused rewrite with one valid solid (IoU 0.94, as recorded by the pipeline); the input surface colored by distance to that delivery on a continuous scale from grey (0) to red (1% of the part size or more); 99% of the surface lies within 0.5%.

Table 2: Study 1, 300 held-out fresh2 parts (after the recovery amendment). Gated IoU of the executed program; v2, v3 and trim are averaged over three seeds within part and their valid counts are given per seed (min–max). Minutes: successful-attempt wall time per part (299 measured per run; three parts usually shared a node; retries excluded), mean over parts and median over parts; for three-seed arms both are averaged over the seeds' per-run values. Failure classes per run are in Table 5.

Arm	Gated IoU	benchA	benchE	Valid / 300	Mean min	Median min
d0 (released)	0.798	0.713	0.882	259	7.5	3.7
dx (+ exec.-verified delivery)	0.901	0.893	0.909	293	16.0	5.5
v2 (+ geometry fixes)	0.930	0.913	0.946	298–299	13.5	6.4
v3 (+ precise extents)	0.936	0.923	0.949	298–300	15.1	7.2
trim (+ silhouette trim)	0.945	0.933	0.957	299–300	partial (see text)	
section-only reference	[pending: section-only control (running)]					
Contrasts (difference, 95% interval, parts better/worse/neutral at ± 0.005, p)						
dx − d0 (primary)	+0.104 [+0.071, +0.138], 43/4/253, $p < 10^{-4}$					
v2 − dx	+0.028 [+0.013, +0.047], 57/20/223, Holm$_3$ $p < 10^{-4}$					
v3 − v2	+0.006 [+0.000, +0.013], 69/38/193, Holm$_3$ p 0.004					
trim − v3	+0.009 [+0.005, +0.013], 90/1/209, Holm$_3$ $p < 10^{-4}$					
trim − section-only	[pending: trim vs section-only (running)]					

4 Study 1: The optimizer on held-out parts

Arms. All arms run frozen source trees on the 300 fresh2 parts. **d0** is the released CADFit pipeline with two common fixes applied to every arm (a memory bound on mesh decimation with a retry of crashed CadQuery workers, and a seeded surface sampler for the revolve-axis search). **dx** adds execution-verified delivery: each candidate program is executed and checked, the best valid one is delivered, and an invalid result is fused or repaired (ShapeFix per invalid solid, dropping a solid that stays invalid) before falling back. **v2** adds geometry fixes to candidate generation and composition; **v3** adds precise extrusion extents (extent sweep, finer steps along thin directions, ends snapped to cap planes with the unsnapped candidate kept); **trim** adds a silhouette-trimmed variant (the best valid program intersected with prisms over the target's three axis silhouettes) as one more candidate for the execution check. d0 and dx run once; v2, v3 and trim run with three seeds, averaged within part. The primary contrast, dx − d0, is analysed alone; the secondary family (Holm over four tests) is v2 − dx, v3 − v2, trim − v3 and trim against a section-only reference (one executed mid-plane section extrusion per axis, best of three).

Execution-verified delivery. The released pipeline obtains a valid replay on 259 of 300 parts. Of the 41 failures, 23 programs build an OCC-invalid solid, 7 raise a build error, 1 produced no program, and 10 exceed the replay limits (time or memory); the last class says nothing about the geometry and is scored 0 under the protocol. Delivering the best program that actually builds valid raises gated IoU from 0.798 to 0.901 (Table 2), a primary contrast of +0.104 [+0.071, +0.138]. The

gain comes from validity: on parts where both arms deliver a valid program the mean difference is +0.0035.

Later stages. Each later stage adds a smaller, significant increment: +0.028 for the geometry fixes, +0.006 for precise extents, and +0.009 for the trim (better/worse 90/1). These adjusted p-values are Holm over the three available tests and are provisional until the fourth (trim against the section-only reference) is computed. The trim records are the trim stage's own replays, which use a 600 s limit instead of the common 900 s; the independent re-judging of every delivered program (pending) applies the common protocol to them. The trim arm also re-runs the fuse/repair pass before trimming. Over all 300 parts and per seed, that pass changes gated IoU by −0.0031 to +0.0029 (the negative value is one seed-0 replay that timed out inside the trim run and is kept as a frozen 0), and the trim itself adds +0.0089 to +0.0096.

Recovery amendment. The first scoring pass stored replay failures that were not evaluations: on one node the sandbox could not be set up (dx: 299 parts), and one v2 seed hit the 900 s limit on 222 parts that the other seeds replayed. After those outcomes were visible we adopted, for all eight runs, a rule giving every setup or resource-limit failure one terminal recovery evaluation with the same judge and limit; programs were not regenerated. It affected 531 records. d0's ten resource failures failed again and stay 0; dx and the v2 seed recovered. On the original records the primary contrast is −0.795 (dx valid on 1 part) and the three-seed v2 mean is 0.699 instead of 0.930. Both analyses are in Appendix C; we report the recovered one as the measurement and label it as amended.

Runtime. Execution checking costs time: mean minutes per part rise from 7.5 (d0) to 16.0 (dx) and 15.1 (v3); per-run medians from 3.7 to 7.2. These are successful-attempt wall times on CPU nodes that usually ran three parts at once; retried attempts are excluded, so they are not total compute. The trim run's timer starts after its initial replay and repair, so its full latency is not measured.

Pending outputs. The section-only reference ([pending: section-only control (running)]), an independent re-judging of every delivered program with a different seed and four times the samples ([pending: independent rejudge (running)]), and surface agreement on the fused boundary ([pending: surface agreement (running)]) are still being computed; no claim here depends on them.

5 Study 2: Frontier agents under the same judge

Harness. An agent receives the target only through tools and edits a model step by step. After every change, the harness assembles the CadQuery program, executes and checks it as in Section 3, and reports build status, validity and executed IoU; the delivered answer is the best executed valid program, or, if no check of a valid program finished within the episode, the current program unverified (it is still re-judged offline like every other delivery). The **code** toolset is `describe_target` (size, volume, flat-face directions and offsets), `section` (exact cross-sections of target and model), `evaluate`, `add_code`, `remove_step`, `list_steps` and `finish`: it gives target inspection and execution feedback, so it is not an unaided language-model baseline. The **CADFit** toolset adds the optimizer's stages: direction detection, sketch extraction on a plane with line/arc fitting, extrusion and revolve fitting, section extrusion, and candidate management. Each episode has 40 tool calls and 30 minutes.

Models and runs. Claude Opus 5.5 (`claude-opus-5-5`) and Sonnet 5.5 (`claude-sonnet-5-5`) run through the Anthropic Messages API with adaptive thinking at effort high, at most 32k output tokens per turn, the full conversation history, and no server-side fallback to another model. Each model runs three independent episodes per part and toolset on the 40 reserved parts; every delivered program is re-judged offline by the same replay and judge as the optimizer. These comparisons were specified after the optimizer study and are exploratory; intervals are nominal.

Results. Opus with the code toolset scores 0.975 with 120/120 valid deliveries (Figure 3, Table 3). On the same parts the optimizer's v3 scores 0.951; the paired difference +0.024 [+0.003, +0.048] excludes zero, while the difference from the optimizer with trim, +0.017 [−0.001, +0.036], does not,

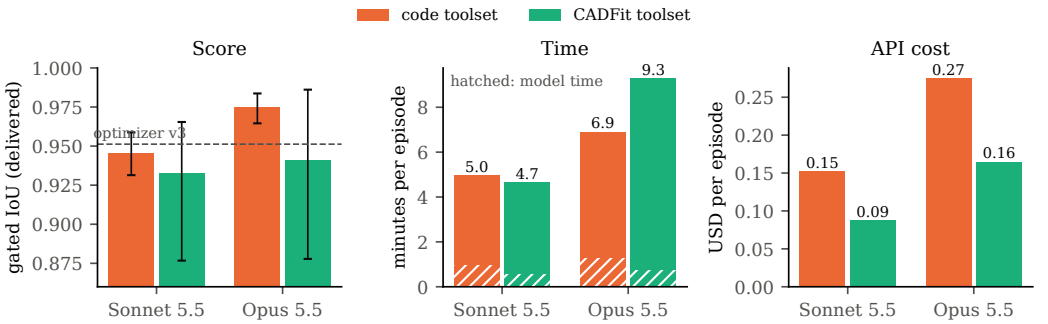


Figure 3: Study 2, 40 reserved fresh2 parts, three episodes per part and condition. Left: gated IoU of the delivered program (mean over parts of the per-part mean; 95% interval); dashed: the optimizer's v3 on the same parts. Middle: mean minutes per episode, hatched part waiting on the model. Right: mean API cost per episode. Runtime and cost are means without intervals.

Table 3: Study 2 against the optimizer on the same 40 parts (gated IoU, 95% interval). Valid: delivered programs that build valid / episodes; no-output episodes ended at the deadline with nothing delivered. Differences are paired over parts (better/worse); exploratory.

	Gated IoU	95% interval	Valid	No output	Mean min	USD / episode
Opus + code	0.975	[0.965, 0.984]	120/120	0	6.9	0.27
Opus + CADFit tools	0.941	[0.878, 0.986]	115/120	5	9.3	0.16
Sonnet + code	0.946	[0.931, 0.959]	120/120	0	5.0	0.15
Sonnet + CADFit tools	0.932	[0.877, 0.965]	117/120	3	4.7	0.09
optimizer d0 / dx	0.779 / 0.934					
optimizer v3 / + trim	0.951 / 0.958	[0.928, 0.968] / [0.940, 0.973]				
Opus code − v3	+0.024 [+0.003, +0.048], 25/15					
Opus code − trim	+0.017 [−0.001, +0.036], 22/18					
CADFit tools − code	Opus −0.034 [−0.097, +0.009]; Sonnet −0.013 [−0.068, +0.020]					

so superiority over the full optimizer package is not established. Opus exceeds Sonnet with the code toolset by +0.029 [+0.018, +0.042], better on 37/2 parts.

The optimizer's stages as tools. With the CADFit toolset, both models score lower on average than with code (Opus −0.034, Sonnet −0.013), and neither interval excludes zero. The difference is a failure tail, not worse programs: all 472 programs the agents delivered are valid, and every failure is an episode that reached its deadline with nothing delivered (5 for Opus, 3 for Sonnet, on three parts). Offline validity is not the same as in-episode verification: 22 valid deliveries had not completed an executed check within their episode.

Where the time goes. Tool execution, not the model, dominates episodes: `add_code`, whose cost is the executed check after every change, takes 84–89% of tool time in the code arms, and single calls of the CADFit stages reached 1776 s. Mean minutes per episode are 6.9 (Opus, code) to 9.3 (Opus, CADFit tools); the optimizer's v3 takes 10.4 minutes per part on these parts, on different hardware and without API cost.

6 Study 3: Removing the tools' overhead

Profiling. On development parts only, we timed every stage inside the tool calls of a scripted policy that uses the CADFit tools. Sketch extraction spent 99.6% of its time (4 development parts) fitting lines and arcs to every profile on the plane; a function-level profile of one development part attributed most of that to two routines: duplicate-point removal by an all-pairs loop, and a greedy segmenter that refits the whole run with an SVD each time it extends by one point. Revolve fitting rotated points one at a time in Python, and every executed check reloaded and rebuilt the target.

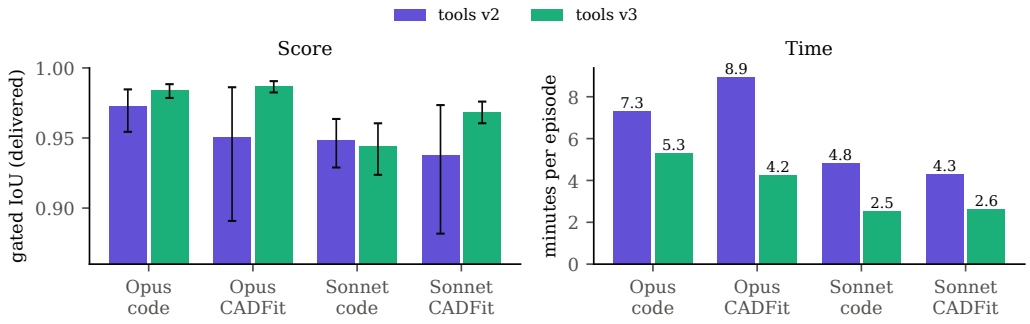


Figure 4: Study 3, 40 fresh3 parts, three episodes per part and condition. Left: gated IoU of the delivered program (95% interval). Right: mean minutes per episode (means without intervals).

Table 4: Study 3 comparisons (gated IoU, paired over parts; better/worse at $\pm 10^{-9}$; sign test p; Holm over the three Opus tests). Below: mean minutes per part, v3 − v2 (descriptive), with parts faster.

Comparison		Difference & 95% interval	Better/worse	p (Holm)
Opus	CADFit v3 − v2 (primary)	+0.037 [+0.002, +0.096]	22/12	0.121 (0.243)
Opus	CADFit v3 − code v3	+0.003 [+0.000, +0.006]	28/11	0.009 (0.028)
Opus	code v3 − v2	+0.011 [+0.001, +0.028]	21/18	0.749 (0.749)
Sonnet	CADFit v3 − v2	+0.031 [−0.001, +0.084]	18/16	0.864
Sonnet	CADFit v3 − code v3	+0.024 [+0.010, +0.045]	28/12	0.017
Sonnet	code v3 − v2	−0.004 [−0.009, −0.000]	15/20	0.500
Minutes, v3 − v2		Opus CADFit −4.7 [−6.6, −2.9]	faster on 33/40	$< 10^{-4}$
		Opus code −2.0 [−3.6, −0.6]	23/40	0.43
		Sonnet CADFit −1.7 [−2.8, −0.7]	25/40	0.15
		Sonnet code −2.3 [−4.0, −0.7]	21/40	0.87

The revised package (tools v3). Four changes are exact rewrites, each checked against the old code on development inputs: KD-tree duplicate removal (identical output on 300/300 randomized cases), the segment error from running sums with the old SVD fit deciding whenever a value lies within 10^{-6} (relative) of a threshold it is compared with (identical fitted output on 162/162 sketches from 8 development parts and 9 planes; fitting time 506 s → 18 s together with the first change), vectorized rotation (bit-identical on 200/200 cases), and loading the target once per process in the judge (18/18 identical); the checks are recorded in `results/tools/equivalence.json`. One change alters behaviour: each executed check gets at most 120 s. A step whose check does not finish stays in the model unchecked and is never delivered while a checked valid program exists. The study tests this package as a whole; it does not separate the rewrites from the cap.

Design. The protocol fixed, before any run and before the cohort was drawn: tools {v2, v3} × toolset {code, CADFit} × {Opus, Sonnet} on the 40 fresh3 parts, three episodes each (960 episodes), same budget and settings as Study 2, each version running from its own frozen tree, delivered programs re-judged offline. The primary comparison is Opus with CADFit tools, v3 − v2; the Opus family (primary, CADFit tools v3 − code v3, code v3 − v2) is Holm-corrected over three sign tests; the same comparisons for Sonnet are reported uncorrected. Runtime summaries were prespecified; timing contrasts and their tests are descriptive. Episodes lost to API credit exhaustion, rate limits, a faulty node and one client timeout crash were set aside and rerun before any analysis (Appendix D).

Results. The primary comparison is positive, +0.037 [+0.002, +0.096], but its declared sign test does not establish an improvement ($p = 0.121$, Holm 0.243); we do not read it as evidence of no effect either (Table 4). With tools v3, Opus delivers a valid program in all 120/120 episodes with the CADFit tools (v2: 116/120, with 1 no-output and 3 invalid), and those tools beat plain code by

+0.003 [+0.000, +0.006], better on 28/11 parts, Holm $p = 0.028$: a small but consistent advantage, the reverse of the sign seen with tools v2 in Study 2. For Sonnet the same contrast is +0.024 [+0.010, +0.045] (uncorrected $p = 0.017$). The best cell, Opus with CADFit tools v3, scores 0.987 [0.983, 0.991]. Across all 960 episodes, 949 delivered programs are valid offline, of which 48 had not completed an in-episode check.

Time. Mean minutes per part with the CADFit tools fall from 8.9 to 4.2 for Opus, faster on 33 of 40 parts, and from 4.3 to 2.6 for Sonnet. In the other cells the mean also falls, but the per-part sign tests are not significant, and the means are dominated by a few parts that used to run long. Opus with code v2 reproduces Study 2 on this new cohort (0.973 here, 0.975 there).

7 Discussion and limitations

What changes when the program is executed. The largest held-out improvement to the optimizer came from no new geometry at all: delivering the program that actually builds valid recovered most of what the search had found. Under the same execution-based judge, a frontier model with inspection, plain code and execution feedback reached the optimizer's level on reserved held-out parts, at a few minutes and cents per part. The optimizer's stages first looked useless as agent tools; profiling located the cost in redundant computation, and with a revised package (exact rewrites plus a per-check time cap) those stages gave the strongest agent a small, consistent edge over plain code. Measuring tools by their cost as well as their output therefore changed the conclusion about them; which part of the package matters was not isolated.

Limitations. (i) The optimizer study's measurement was amended after outcomes were visible (one terminal recovery evaluation for setup and resource-limit replay failures); the original analysis is reported beside it, and it changes the dx and v2 values substantially. (ii) The agent studies use 40 parts per cohort with three episodes each, so many intervals are several hundredths of IoU wide, and the Study 2 comparisons are exploratory. (iii) One benchmark: hard-tier CADBench parts from two splits. (iv) The endpoint is volumetric IoU; surface agreement and an independent re-judging are pending. (v) The code toolset includes target inspection and execution feedback, so it measures an agent in this harness, not a language model alone. (vi) Closed-model results are tied to specific model versions, settings (32k tokens per turn, full history) and an API; the tools-v3 package bundles exact rewrites with a behavioural check cap, and its output-equivalence was checked on development inputs only. (vii) Infrastructure failures (faulty nodes, API limits) required reruns, listed in Appendix D.

Reproducibility statement

Every result value in the text and tables is a macro generated from committed result files by `bench/paper_numbers.py`, and every figure is drawn by a committed script; configuration constants, cohort dates and the counts of the development example in Figure 2 come from the protocols, the code and the pipeline's records. The protocols, with their dated amendments and execution logs, the frozen source trees of every arm, the cohort definitions (seeded draws excluding earlier model indices), the scoring and analysis code, and the per-part and per-episode records will be released with the paper. Closed-model episodes require an Anthropic API key; their raw records are included, so the analyses can be re-run without the API.

References

CadQuery contributors. CadQuery: A python parametric CAD scripting framework based on OCCT. `https://github.com/CadQuery/cadquery`, 2024.

Anna C. Doris, Jacob Thomas Sony, Ghadi Nehme, Era Syla, Amin Heyrani Nobari, and Faez Ahmed. Cadbench: A multimodal benchmark for ai-assisted cad program generation, 2026. URL `https://arxiv.org/abs/2605.10873`.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

Tao Hu, Jiaxin Ai, Licheng Wen, Xueheng Li, Shu Zou, Siqi Li, Nianchen Deng, Xinyu Cai, Hongbin Zhou, Pinlong Cai, Daocheng Fu, Yu Yang, Hairong Zhang, Botian Shi, and Xuemeng Yang. Itercad: An iterative multimodal agent for visually-grounded cad generation and editing, 2026. URL https://arxiv.org/abs/2606.13368.

Alec Jacobson, Ladislav Kavan, and Olga Sorkine-Hornung. Robust inside-outside segmentation using generalized winding numbers. *ACM Transactions on Graphics*, 32(4), 2013. doi: 10.1145/2461912.2461916.

Soslan Kabisov, Vsevolod Kirichuk, Andrey Volkov, Gennadii Savrasov, Marina Barannikov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadreasoner: Iterative program editing for cad reverse engineering, 2026. URL https://arxiv.org/abs/2603.29847.

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning, 2019. URL https://arxiv.org/abs/1812.06216.

Emmett Lambert and Manifold contributors. Manifold: geometry library for topological robustness. https://github.com/elalish/manifold, 2024.

Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: Tool-augmented vllms as generic cad task solvers, 2025. URL https://arxiv.org/abs/2412.13810.

Ghadi Nehme, Eamon Whalen, and Faez Ahmed. Cadfit: Precise mesh-to-cad program generation with hybrid optimization, 2026. URL https://arxiv.org/abs/2605.01171.

Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds, 2025. URL https://arxiv.org/abs/2412.14042.

Gennadiy Savrasov, Maksim Elistratov, Nikita Gavrilov, Albert Garifullin, Oleg Pavlov, Soslan Kabisov, Vladimir Frolov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadfather: Autonomous cad reconstruction through coordinated tool use, 2026. URL https://arxiv.org/abs/2610.09127.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Mikaela Angelina Uy, Yen yu Chang, Minhyuk Sung, Purvi Goel, Joseph Lambourne, Tolga Birdal, and Leonidas Guibas. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders, 2022. URL https://arxiv.org/abs/2112.09329.

Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences, 2021. URL https://arxiv.org/abs/2010.02392.

Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models, 2021. URL https://arxiv.org/abs/2105.09492.

Xianghao Xu, Wenzhe Peng, Chin-Yi Cheng, Karl D. D. Willis, and Daniel Ritchie. Inferring cad modeling sequences using zone graphs, 2021. URL https://arxiv.org/abs/2104.03900.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

A COHORTS

All cohorts are seeded draws from the CADBench index (hard tier) by `bench/fetch_cadbench.py`, taken in a fixed order so that adding a cohort never changes an earlier one, and excluding any part whose model index (the first field of its file id) appears in an earlier set. The exclusion is by model index only: two fresh3 parts share the second file-id field (a document hash) with a fresh2 part (00078986 with 00078985, 00894209 with 00894237), so near-relatives of two fresh2 parts may appear in fresh3. *beamfix5* is five parts kept in `bench/cadbench/beamfix5`. Development used *test* (100 benchA), *dev* (30 + 30), *smoke* (5), *beamfix5* (5) and *fresh* (30 + 30). *fresh2* (150 + 150) was drawn on 2026-10-18 after the optimizer repairs were developed. The 40 reserved parts were drawn on 2026-11-01 from fresh2's manifest alone (seed 20261101, 20 per split) before any fresh2 outcome. *fresh3* (20 + 20) was drawn on 2026-11-06 after the tools-v3 code and protocol were committed; it shares no model index with any earlier set.

B FAILURE CLASSES

Table 5: Study 1 delivery outcome per part and run (300 parts each), after the recovery amendment and, in brackets, on the original records where they differ.

Run	Valid	Built invalid	Build error	Missing	Resource limit	Sandbox setup
d0	259	23	7	1	10	0
dx	293 [1]	1 [0]	0	0	0	6 [299]
v2 seed 0	299	1	0	0	0	0
v2 seed 1	298	2	0	0	0	0
v2 seed 2	298 [77]	2 [1]	0	0	0 [222]	0
v3 seed 0	300	0	0	0	0	0
v3 seed 1	300	0	0	0	0	0
v3 seed 2	298	1	0	1	0	0

For the agent studies, delivered-valid counts and no-output episodes are in Table 3; in Study 3 the only invalid deliveries were 3 (Opus, CADFit tools v2), 1 (Opus, code v2), 1 and 1 (Sonnet, code v2 and v3) and 1 (Sonnet, CADFit tools v2); no-output episodes: 1 (Opus, CADFit v2) and 3 (Sonnet, CADFit v2), none with tools v3.

C RECOVERY AMENDMENT: ORIGINAL AND RECOVERED ANALYSES

Eligibility (decided 2026-11-05, after the affected score files existed, applied to all eight runs): a replay failure whose cause is a sandbox that could not be set up (*setup*; no evaluation took place) or an rc -9 kill or the 900 s limit (*resource*; machine or program). Each eligible part received one recovery evaluation with the same judge, limit and the scorer's existing single retry of a silent replay death, at fixed concurrency; the result is terminal, and a repeated failure scores 0. Programs were not regenerated and no search or selection was redone. Affected records: 299 dx (setup), 222 v2 seed 2 (resource), 10 d0 (resource); total 531.

Table 6: Gated IoU on the original records and after recovery (300 parts).

	Original	Recovered
d0	0.798	0.798
dx	0.002	0.901
v2 seed 2	0.238	0.930
v2, three-seed mean	0.699	0.930
dx − d0 (primary)	−0.795	+0.104

D EXCLUSIONS, RERUNS AND COST

Events not visible in the result files are recorded in the execution logs of the two protocols (`review-stage/HELDOUT_PROTOCOL.md`, `review-stage/TOOLS_V3_PROTOCOL.md`). **Study 1.** Pipeline chunks that failed for infrastructure reasons (nodes without the software stack, out-of-memory workers) were resubmitted under the same frozen trees as recorded in the protocol's execution log; the first dx recovery job was scheduled on a node whose sandboxed replays timed out elsewhere and was cancelled before writing results, then resubmitted excluding that node. **Study 2.** A first Opus run on one node was set aside: 43 of its 44 diagnosed episodes contain executed-call timeouts at 90 s (578 in total); 35 episodes wrote records, with $8.95 recorded cost (the cost of the rest is unknown). It was rerun on other nodes, and every later job first replays a box through the sandboxed path and stops before any API call if that fails. Retained Study 2 episodes cost $81.40. **Study 3.** 239 episodes ended on API errors (credit exhaustion and rate limits; $7.44 recorded) and were moved aside and rerun; one job stopped at its preflight on a faulty node (53 episodes not started) and one episode crashed at its deadline when the frozen client did not catch a streaming read timeout (no record written); both were rerun from the same frozen tree. Retained Study 3 episodes cost $154.96.

E HARNESS AND SETTINGS

Agents see the target only through tools; frames are normalized (largest extent spans $[-1, 1]$). Tool replies report the executed program's build status, validity and IoU after every change; the delivered answer is the best executed valid program (valid first, then IoU). Claude runs use the official SDK (configuration in `agent/anthropic_chat.py` and `bench/claude_job.sh`) with streaming, adaptive thinking, effort high, a 32k-token limit per turn, prompt caching, the full native message history (tool results are not truncated), and no server-side fallback; a refusal or API error ends an episode. Episodes run 3–8 at a time per CPU node (the `-jobs` setting of each submission). In Study 3 the two tool versions are identified by the frozen source tree each job ran from (output paths under `tv_v2_src` and `tv_v3_src`); the episode records' `tools` field names the harness mode, which is the same for both.